

Introduction to Automata Theory

>>Find out what it takes to program an automaton, apart from the four main components of its component theory> by Ashesh Lal

Automata theory is the study of abstract machines (like finite state machines), automata (self-operating machines) and the computational problems that can be solved using them. It comes under the broad topic of Theoretical Computer Science, under Discrete Mathematics. Essentially, Automata Theory is the study of self-operating virtual machines to help with the logical understanding of input and output processes, with or without intermediate stage(s) of computation/function/process. Automata play a major role in theory of computation, compiler design, artificial intelligence, parsing and formal verification and have been around since ancient times.

Automata (pl.) are defined to study useful machines under mathematical formalism. So, the definition of an automaton is open to variations according to the “real world machine”, which we want to model using the automaton (sg.).

There are four main components of any automata that define its variant:

1. Input
 - a. Finite input: An automaton that accepts only finite sequence of symbols.
 - b. In finite input: An automaton that accepts infinite words (ω -words). Such automata are called ω -automata.
2. States
 - a. Finite states: An automaton that contains only a finite number of states.
 - b. Infinite states: An automaton that may not have a finite number of states, or even a countable (http://en.wikipedia.org/wiki/Countable_set) number of states. For example, the quantum finite automaton or topological automaton has uncountable infinity of states.
3. Transition function
 - a. Deterministic: For a given current state and an input symbol, if an automaton can only jump to one and only one state.
 - b. Nondeterministic: An automaton that, after reading an input symbol, may jump into any of a number of states, as licensed by its transition relation, i.e. the automaton non-deterministically decides to jump into one of the allowed choices.

4. Acceptance condition

We will be primarily concentrating on finite or countable infinite input on finite state machines in the rest of this article.

Deterministic Finite Automata (DFA) and Non-deterministic Finite Automata (NFA) have the same computational power and both are capable of recognizing the same languages, called the regular languages that regular expressions can describe. A tutorial on regular expressions can be found on <http://regexone.com/>.

Now if we take the language to be $L = (a|b)^*abb$. Then L will be the set of strings that are accepted by this regular expression, which is read as “a or b, zero or many times, followed by the string abb” or “all the strings in the alphabet $\{a,b\}$ that end in abb”.

Language $L = (a|b)^*abb = \{abb, aabb, babb, aaabb, ababb, baabb, bbabb, \dots\}$

superscript (*) - Kleene closure - zero or many
superscript (+) - Positive closure - one or many
 ϵ - Epsilon - empty string

The NFA for this language L is depicted in Fig. 1. States are represented by numerals in circles and the transition between states is described by alphabets over the arrow. The start state is 0 and the end state (or accepting state) is 10, which is denoted by the double-circle. Sometimes the end state is realized by making the said state accept the transition labeled #.

An NFA accepts input string x if and only if there is a path in the transition graph from the start state (0) to one of the accepting states,

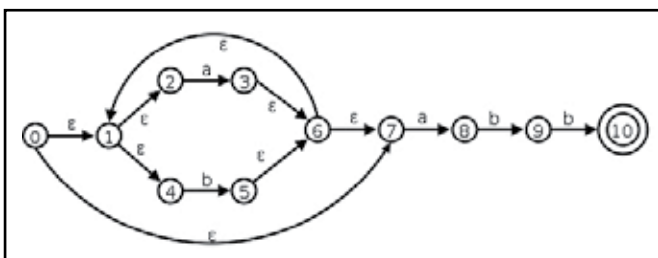


Fig.1 This transition graph is an NFA for the regular expression $(a|b)^*abb$

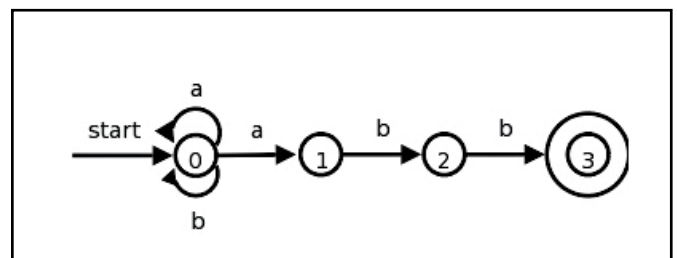


Fig. 3 This is the transition graph for the regular expression $(a|b)^*abb$